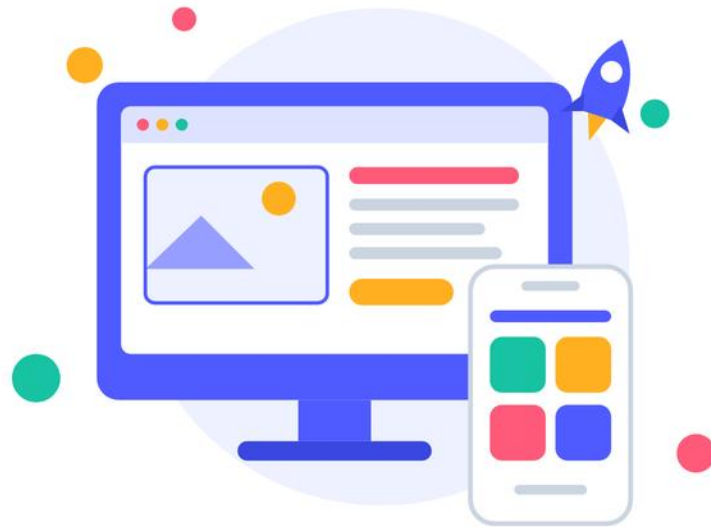


WEEK 03 · REFERENCE

THE CSS CATALOGUE

Optional depth, not examined. The catalogue — every selector, unit and colour function — plus every lecture slide the five cycles replaced, and the depth behind each number the cycles measured. Forty-eight slides you will still open in week 10, when a component needs a state and you want the selector in front of you.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



THIS IS YOUR PAGE, RIGHT NOW: VALID, AND UNSTYLED



► **SAY IT OUT LOUD** — this page is **CORRECT**. Valid, accessible, semantic. And nobody would look at it twice.

A STYLE SHEET IS A LIST OF RULES

selector `h1` — which elements this applies to.

declaration block everything between the braces.

declaration one `property: value;` pair. Three of them here.

That is the entire syntax. Everything difficult about CSS is about which rule wins and what a value means — never about the punctuation.

```
1 h1 {  
2   color: hsl(258 72% 42%);  
3   font-size: 2rem;  
4   line-height: 1.2;  
5 }
```

REFERENCE · THREE WAYS, ONE ANSWER

THREE WAYS TO ATTACH CSS

```
1  <!-- 1. inline: on the element itself -->
2  <h1 style="color: teal">Ruppin Coffee Club</h1>
3
4  <!-- 2. internal: a <style> block in the head -->
5  <head>
6    <style>
7      h1 { color: teal; }
8    </style>
9  </head>
10
11 <!-- 3. external: a separate file, linked -->
12 <link rel="stylesheet" href="styles.css" />
```

All three render identically. Only one of them survives contact with a second page.

CHANGE THE BRAND COLOUR. COUNT THE EDITS.

demos/01-three-ways/01-inline — every decision, eight times

↻ reload

open ↗

Ruppín Coffee Club

Third floor, next to the printer that never works.

This week's beans

Ethiopia, Guji. Tastes of stone fruit, and of someone else paying.

Opening hours

Run by second-years who should have been revising.

► **HANDS-ON** demos/01-three-ways/ — Change teal to red in 01-inline/. Count out loud. Then do the same in 03-external/.

WHY EXTERNAL, IN THREE SENTENCES

ONE FILE, MANY PAGES

Every page points at the same stylesheet. A change lands everywhere at once, and two pages cannot drift apart.

THE BROWSER CACHES IT

It is downloaded once for the whole site, not re-sent inside every page.

STRUCTURE STAYS STRUCTURE

The markup goes back to saying what things ARE. That is the file you spent last week getting right.

External CSS lets you use one stylesheet for several different pages.

ONE THING YOU MAY HAVE BEEN TOLD, THAT IS WRONG

WRONG

"Specificity goes: **inline > internal > external**."

It appears in a lot of course material, including an older version of this one.

RIGHT

Where a rule lives is not specificity at all. An internal rule and an external rule with the same selector are decided by **source order** — whichever the browser reads last.

Move the `<link>` below the `<style>` block and the answer flips.

Nothing about "internal" made it stronger.

THE CASCADE, IN ORDER

1 Origin and importance

browser stylesheet, then the user's, then yours. Yours wins — unless somebody wrote `!important`, which reverses that order.

2 Element-attached

`style=""` on the element beats any selector in any sheet — the one place placement matters.

3 Layers

`@layer`, if you declared any: later layers win, unlayered rules beat all layers.

4 Specificity

the triple, column by column.

5 Source order

last one wins. Only reached on an exact tie.

Internal versus external is NOT a step. A `<style>` block and a linked file are decided by source order, like any other tie.

THE CASCADE IN FULL — CSS CASCADE LEVEL 5

1 Origin and importance

eight buckets, strongest first: transitions · `!important` browser, user, author · animations · normal author, user, browser.

2 Element-attached styles

the `style` attribute on the element.

3 Layers

`@layer reset, base, components;` — later wins, unlayered beats all.

Specificity

4 , then order of appearance

.

So `!important` is not "step two of five": it flips the origin order. Two `!important` author rules are then decided by specificity like any other pair — which is why one never ends the argument.

COMPUTING THE TRIPLE

- a** `#id`
- b** `.class` · `[attribute]` · `:pseudo-class`
- c** element names · `::pseudo-element`

`*`, combinators and `:where()` count nothing. `:is()`, `:not()` and `:has()` take the specificity of their strongest argument.

```
1 a /* (0,0,1) */  
2 nav a /* (0,0,2) */  
3 a.button /* (0,1,1) */  
4 a:hover /* (0,1,1) */  
5 #cta /* (1,0,0) */
```

`a.button` and `a:hover` tie. That tie is why hover states so often "stop working".



COMPUTE IT YOURSELF

demos/03-specificity — type any two selectors ↶ reload open ↷

SPECIFICITY CALCULATOR

one id vs five selectors four elements vs one class a tie — so order decides a pseudo-element earns c :where() is free an attribute weighs like a pseudo-class

Selector A

#cta

(1 , 0 , 0) 1 selector counted

a #cta

Selector B

.site .nav .list .item a

(0 , 4 , 1) 5 selectors counted

b .site b .nav b .list b .item c a

A wins: (1,0,0) beats (0,4,1) — even though B has more pieces in it.

► **HANDS-ON** [demos/03-specificity/](#) — Six presets, every one a case where counting gives the wrong answer. Vote before pressing.

SPECIFICITY, WORKED

```
* (0,0,0)
p (0,0,1)
nav p (0,0,2)
nav > ul > li (0,0,3)
.lede (0,1,0)
p.lede (0,1,1)
[type='email'] (0,1,0)
input:required (0,1,1)
a:hover (0,1,1)
.nav .item a (0,2,1)
#page-title (1,0,0)
#nav .item a (1,1,1)
p::first-line (0,0,2)
:where(.card) p (0,0,1)
:is(.a, #b) p (1,0,1)
:not(.a, #b) (1,0,0)
li:has(> a.cta) (0,1,2)
nav a[aria-current] (0,1,2)
```

a

#id

b

.class · [attr] · :pseudo-class

c

element · ::pseudo-element

Compared column by column, like a version number. **b** never carries into **a**, however many of them there are.

`:where()` is always zero — the way to ship a default that anything can override. `:is()`, `:not()` and `:has()` take the specificity of their most specific argument; the pseudo-class itself adds nothing.

THE CASCADE, SHOWN ON THE PAGE

[demos/02-cascade/02-explained](#) — click an element, pick a property [↶ reload](#) [open ↷](#)

This heading is supposed to be teal

This callout is supposed to be blue. Someone edited it. Twice.

Everything in this box is red, because the box says so.

Everything except this button:

Order a flat white

CASCADE INSPECTOR
Click any element above. Then pick a property.
Property

► **HANDS-ON** [demos/02-cascade/02-explained/](#) — Click the heading with `color` selected. Two rows, sorted the way the browser sorts them.

THREE KEYWORDS WORTH KNOWING

```
1  /* take the parent's value, even for a property that does not inherit */
2  .card p { margin: inherit; }
3
4  /* back to the CSS specification's default, ignoring the browser's stylesheet */
5  button { all: initial; }
6
7  /* inherit if the property inherits, otherwise initial */
8  .reset { border: unset; }
```

`inherit` is the useful one this week — `font: inherit` on form controls is cycle 2. The other two matter when you fight a framework — week 6.

WHAT INHERITS, AND WHAT DOES NOT

INHERITS

color font font-family
font-size font-weight
line-height letter-spacing
text-align text-transform
white-space visibility
cursor list-style direction

DOES NOT

margin padding border
background width height
display position overflow
box-sizing opacity
transition

FORCE IT EITHER WAY

inherit take the parent's value.
initial the specification's default.
unset inherit if it inherits,
otherwise initial.
revert back to the browser's own
stylesheet.

EVERY SELECTOR, ON ONE PAGE

BASIC

* · p · .class · #id

ATTRIBUTE

[type] · [type='email'] ·
[href^='https'] ·
[href\$='.pdf'] · [class*='card']

LOGICAL

:is() · :where() · :not() · :has()

COMBINATORS

A B · A > B · A + B · A ~ B

STATE

:hover · :focus · :focus-visible ·
:active · :disabled · :checked ·
:required · :invalid

PSEUDO-ELEMENTS

::before · ::after ·
::first-line · ::marker ·
::selection · ::placeholder

GROUPING

h1, h2, h3

POSITION

:first-child · :last-child ·
:nth-child(2n) · :first-of-type ·
:only-child · :empty

REFERENCE · CHOOSING WHAT TO STYLE

TYPE, CLASS, ID

```
1  /* type: every one of them, everywhere */
2  p { line-height: 1.6; }
3
4  /* class: the ones you marked. Reusable. */
5  .lede { font-size: 1.33rem; }
6
7  /* id: exactly one element on the page */
8  #page-title { color: grey; }
```

Type selectors are the default, and last week's work is what makes them enough.

header, nav, table caption, form fieldset — all reachable by what they ARE.

Use a class when there is no element that means it. "The opening paragraph" has no tag. That is what .lede is for.

Never style by id. It cannot be reused, and at (1,0,0) nothing you write later can override it.

COMBINATORS: RELATIONSHIPS BETWEEN ELEMENTS

```
1  /* descendant: any p anywhere inside nav */
2  nav p { margin: 0; }
3
4  /* child: only a direct child */
5  nav > ul { padding-left: 0; }
6
7  /* adjacent sibling: the p immediately after an h2 */
8  h2 + p { font-size: 1.2rem; }
9
10 /* general sibling: every p after an h2, same parent */
11 h2 ~ p { color: grey; }
```

The space is a combinator too, and it is the one people forget: `nav p` and `nav.p` are completely different selectors.

GROUPING, AND HOW FLAT SHOULD A SELECTOR BE

TOO DEEP

```
body main section
  article div p a {
    color: teal;
  }
```

Fragile: one new wrapper and it stops matching. And at (0,0,7) it is hard to override.

FLAT

```
h1,
h2,
h3 {
  line-height: 1.18;
}

a { color: teal; }
```

Says what it means, applies everywhere, easy to override on purpose.

A long chain is almost always somebody raising specificity until something won, instead of finding out what was winning.

REFERENCE · EVERYTHING IS A BOX

FOUR RINGS, AND ONLY ONE OF THEM IS YOURS

content

the text or the image itself.

padding

space INSIDE the border. Takes the background.

border

the edge. Has width, style and colour.

margin

space OUTSIDE the border. Always transparent.

Padding and margin look identical on a white page and behave completely differently the moment there is a background colour or a border.

Padding is part of the box. Margin is the space around it. That sentence answers most of the questions this section produces.



REFERENCE · MOVED FROM THE LECTURE

THE BOX, MEASURED BY THE BROWSER

[demos/04-box-model/03-visualiser](#) — nothing here is simulated [reload](#) [open](#)

THE DECLARATION

box-sizing
content-box (the default)

width 200px

padding 20px

border-width 4px

margin 16px

content

padding

border

margin

THE BOX, IN A 300PX RAIL



```
box-sizing: content-box

width is the CONTENT only, so padding and border are ADDED:

content      200
+ padding x2  40
+ border  x2   8
= box on page 0   (you asked for 200)

+ margin x2   32
= space taken 32   (margin is outside the box, always)
```

Fits the 300px rail, with 264px to spare.

► **HANDS-ON** [demos/04-box-model/03-visualiser/](#) — width 200, padding 20, border 4. Ask how wide the box is BEFORE reading the number.

TWO BOXES THAT WERE TOLD TO BE 240 WIDE

demos/04-box-model/01-overflow – both escape a 260px container

↻ reload

open ↗

TWO BOXES THAT WERE ASKED TO BE 240PX WIDE

The dashed containers are 260px. Both children were given width: 240px.

Ethiopia, Guji

Stone fruit. Someone else paying.

Email

you@ruppin.ac.il

measure them

Guess first. Then press the button.

► **HANDS-ON** demos/04-box-model/01-overflow/ — Press measure them. Neither is 240.

BOX-SIZING: A HISTORY, AND WHY THE DEFAULT IS THE WRONG ONE

1996 – CSS1

`width` is the width of the *content*, padding and border are added outside it. That is `content-box`, and it is still the default in every browser today.

1997–2001 – THE "IE BOX MODEL"

Internet Explorer 5 measured `width` as the whole box — what most people expect. Standards mode fixed IE 6 to the spec; quirks mode kept the old behaviour.

2012 → TODAY

`box-sizing: border-box` (CSS Basic UI Level 3) gives you that measurement on purpose. Paul Irish's 2012 post made the universal reset a habit.

```
/* the course's three lines */
*, ::before, ::after { box-sizing: border-box; }

/* the inheriting variant: a component can opt out for its subtree */
html { box-sizing: border-box; }
*, ::before, ::after { box-sizing: inherit; }
```

MARGINS COLLAPSE. THIS IS NOT A BUG.

```
1 h2 { margin-bottom: 32px; }  
2  
3 p { margin-top: 16px; }  
4  
5 /* the gap between them: 32px, not 48px */
```

Two vertical margins that meet do not add up — the larger one wins. It only happens **vertically**, and only between margins with nothing between them: no border, no padding, no content.

The rules: only block-axis margins; adjacent siblings; a parent and its first or last child when nothing separates them (no border, padding, inline content, or a new formatting context); an empty block's own top and bottom. Two positives: the larger wins. A negative and a positive: they add. Flex and grid items never collapse.

NOT EVERY BOX TAKES A WIDTH

BLOCK BOXES

`div` `p` `h1` `section` `ul` `li`

Start on a new line, take the full width available, accept `width` and `height`.

INLINE BOXES

`a` `span` `strong` `em` `code` `label`

Sit inside a line of text. `width` and `height` **do nothing to them**, and vertical padding does not push neighbours away.

"I set a width on my link and nothing happened" is not a bug. Next week's `display` is the property that changes which kind of box an element makes.

THE FOUR SHORTHANDS YOU WILL ACTUALLY TYPE

```
1  /* one value: all four sides */
2  padding: 16px;
3
4  /* two: vertical, then horizontal */
5  padding: 16px 24px;
6
7  /* four: clockwise from the top – top, right, bottom, left */
8  padding: 8px 16px 24px 16px;
9
10 /* border and radius follow the same idea */
11 border: 1px solid hsl(220 20% 80%);
```

Clockwise from the top. TRouBLE, if a mnemonic helps.

THE BOX, AND ITS SHORTHANDS

```
margin: 16px;                /* all four */
margin: 16px 24px;           /* vertical | horizontal */
margin: 8px 16px 24px;       /* top | horizontal | bottom */
margin: 8px 16px 24px 32px;  /* top right bottom left, clockwise */
margin: 0 auto;              /* centre a block with a max-width */

padding: 16px;               /* same four patterns */

border: 1px solid hsl(220 20% 80%); /* width style colour */
border-bottom: 2px solid;           /* colour: currentColor */
border-radius: 12px;                /* four corners */
border-radius: 12px 12px 0 0;       /* TL TR BR BL */

outline: 3px solid;              /* like a border, but takes NO space */
outline-offset: 2px;

box-sizing: border-box;          /* width includes padding and border */
```

LOGICAL PROPERTIES: START AND END, NOT LEFT AND RIGHT

```
/* physical          logical          */
margin-left: 1rem;   margin-inline-start: 1rem;
margin-top: 1rem;    margin-block-start: 1rem;
padding: 0 1rem;     padding-inline: 1rem;
border-left: 2px solid; border-inline-start: 2px solid;
text-align: left;    text-align: start;
width: 65ch;         inline-size: 65ch;
```

Inline is the direction text runs; *block* is the direction lines stack. In English, inline-start is the left edge. In Hebrew — `dir="rtl"` — it is the right edge, and every "left" you typed is now on the wrong side.

A rule like `border-left` — the one down the nav in this week's finished page — is written `border-inline-start` and moves to the right edge of a Hebrew page on its own. This course's PDFs and the slide theme are written that way.

Supported everywhere since 2021.

REFERENCE · VALUES

ONE COLOUR, THREE NOTATIONS

```
1 /* hex: two hex digits per channel. Compact, unreadable. */  
2 color: #4c1eb8;  
3  
4 /* rgb: how much of each light. Readable, still not meaningful. */  
5 color: rgb(76 30 184);  
6  
7 /* hsl: hue, saturation, lightness. Meaningful. */  
8 color: hsl(258 72% 42%);
```

Identical pixels. Now make it 10% lighter — and notice that only one of the three lets you do that without a colour picker.

A WHOLE PALETTE FROM ONE NUMBER

demos/06-colour/hsl-explorer – drag the hue across the range

↻ reload

open ↗

ONE COLOUR, THREE NUMBERS

hue 234



saturation 89%



lightness 52%



■ hsl hsl(234 89% 52%)
rgb rgb(24 45 242)
hex #182df2

THE WHOLE PALETTE, DERIVED FROM THAT ONE HUE

Every row is the same hue. Only saturation and lightness move.

	token	hsl	hex	on surface
	--color-surface	hsl(234 40% 98%)	#f8f8fc	1.00:1 – this is the surface
	--color-border	hsl(234 30% 82%)	#c3c6df	1.59:1 – decorative – no minimum
	--color-border-strong	hsl(234 34% 48%)	#5159a4	5.99:1 AA
	--color-accent	hsl(234 89% 52%)	#182df2	7.36:1 AA
	--color-accent-strong	hsl(234 89% 38%)	#0b1cb7	10.65:1 AA
	--color-text-muted	hsl(234 20% 34%)	#454968	8.26:1 AA
	--color-text	hsl(234 47% 11%)	#0f1229	17.48:1 AA

Drag the hue towards yellow and watch the accent row go red. Same lightness, same saturation, failing contrast — because HSL's "lightness" is a number in a colour model, not how bright the colour looks to an eye. Yellow at 52% is far brighter than blue at 52%. This is why the ratio is checked and never assumed.

Ruppín Coffee Club

Body text on the surface colour, at the ratio in the table.

[A link, in the accent colour](#)

► **HANDS-ON** demos/06-colour/hsl-explorer/ — Drag HUE end to end. The palette stays coherent at every stop. Then ask them to do that with hex.

THE TRAP INSIDE HSL

SAME LIGHTNESS, DIFFERENT BRIGHTNESS

`hsl(240 90% 52%)` — blue — passes AA on white.

`hsl(60 90% 52%)` — yellow — fails badly.

Both say 52%.

WHY

"Lightness" in HSL is a coordinate in a colour model, not how bright a colour looks to an eye. Human vision is far more sensitive to green and yellow than to blue.

So: never assume. Check.

COLOUR

```
color: #4c1eb8;           /* hex, 6 digits          */
color: #4c1eb880;         /* hex, 8 digits: + alpha */
color: #abc;              /* shorthand for #aabbcc  */
color: rgb(76 30 184);    /* modern, space-separated */
color: rgb(76 30 184 / 50%); /* with alpha             */
color: hsl(258 72% 42%);  /* hue, saturation, light  */
color: hsl(258 72% 42% / 0.5); /* with alpha, after a slash */
color: hsl(258, 72%, 42%); /* legacy commas: still valid */
color: hsl(var(--hue) 72% 42%); /* a token inside a function */
color: oklch(52% 0.22 290); /* perceptual lightness (2023+) */
color: rebeccapurple;     /* 148 named colours exist */
color: currentColor;      /* whatever `color` is here */
color: transparent;
```

`currentColor` is the useful one nobody teaches: a border that always matches the text without repeating the value. The space-separated form with `alpha` is the modern syntax (CSS Color 4); `hsla()` is an alias.

CONTRAST, AND WHERE TO CHECK IT

1.4.3 · AA · 4.5:1

Text and images of text. The threshold that matters, and the one most designs fail. Cycle 3 measured 2.55 before and 5.45 after.

1.4.3 · AA · 3:1 FOR LARGE TEXT

Large = 18pt (24px) regular, or 14pt (18.66px) bold. 1.4.11 asks the same 3:1 of UI boundaries and focus rings against their background.

1.4.6 · AAA · 7:1

4.5:1 for large. Rarely required, always welcome. Logos and disabled controls are exempt at every level.

webaim.org/resources/contrastchecker · color.adobe.com · colorhunt.co

DevTools shows the ratio in the colour picker itself.

FONT STACKS: A LIST, IN ORDER OF PREFERENCE

```
1 font-family:  
2   system-ui, -apple-system, 'Segoe UI', Roboto,  
3   'Helvetica Neue', Arial, sans-serif;  
4  
5 /* the last name is the generic family – always end with one */  
6 font-family: Georgia, 'Times New Roman', serif;
```

`system-ui` is whatever the reader's own operating system uses. Already installed, already tuned for their screen, downloads nothing.

WEB FONTS, AND WHAT THEY COST

```
1 @font-face {  
2   font-family: 'Heebo';  
3   src: url('/fonts/heebo.woff2') format('woff2');  
4   font-weight: 400;  
5   font-display: swap;  
6 }  
7  
8 /* now it is just another name in the stack */  
9 body { font-family: 'Heebo', system-ui, sans-serif; }
```

Host the file yourself. A font from someone else's CDN is a request to a third party on every page load, a dependency on their uptime, and nothing at all in an exam room with no network.



TYPE

```
/* the system stack: installed, tuned, downloads nothing */
font-family: system-ui, -apple-system, 'Segoe UI', Roboto, Arial, sans-serif;

/* always end with a generic family */
font-family: Georgia, 'Times New Roman', serif;
font-family: ui-monospace, 'SF Mono', Menlo, Consolas, monospace;

/* the shorthand: style weight size/line-height family – size and family required */
font: 700 1.25rem/1.2 var(--font-sans);

/* self-hosted, because a CDN is a dependency you did not choose */
@font-face {
  font-family: 'Heebo';
  src: url('/fonts/heebo.woff2') format('woff2');
  font-display: swap;
}
```

TWO NUMBERS THAT DECIDE WHETHER TEXT IS READABLE

LINE-HEIGHT

`1.5` to `1.7` for body text. Unitless, so it multiplies the element's own size and keeps working when the size changes.

Headings want `1.1` to `1.2` — at 2rem, body leading opens a canyon between two lines of one heading.

MEASURE

`max-width: 65ch`. One `ch` is the width of a `0` in the current font, so this says "about 65 characters".

A line running the full width of a wide monitor is genuinely hard to read: the eye loses its place coming back.

A TYPE SCALE IS MULTIPLICATION, NOT OPINION

```
1  /* base 17px, each step x1.25 */
2  --font-size-base: 1.0625rem;
3
4  --font-size-sm:    0.85rem;    /* base / 1.25 */
5  --font-size-lg:    1.33rem;    /* base x 1.25 */
6  --font-size-xl:    1.66rem;    /* x 1.25 again */
7  --font-size-2xl:   2.07rem;    /* and again    */
```

Four sizes chosen by one ratio look related. Four sizes chosen one at a time look like four decisions, because they are.

EM COMPOUNDS. REM DOES NOT.

demos/05-units — one rule, five nesting levels

↻ reload

open ↗

ONE RULE: LI { FONT-SIZE: 0.9EM }

Drinks

Coffee

Filter

Ethiopia

Guji, washed

measure every level

Guess the size of the deepest item first.

► **HANDS-ON** [demos/05-units/](#) — Ask for the size of the deepest item. Press. Then open [02-rem/](#) and drag the reader's font-size slider.

EM AND REM, THE ARITHMETIC

```
html { /* 16px unless the reader changed it */ }
body { font-size: 1.0625rem; } /* 16 × 1.0625 = 17px */
h1   { font-size: 2.07rem; } /* 16 × 2.07 = 33.12px */
      /* NOT 17 × 2.07 */

li { font-size: 0.9em; } /* × the PARENT's size */
/* li in li in li: 16 × .9 × .9 × .9 = 11.66px */

p { line-height: 1.6; } /* unitless: × each own size */
```

rem

the root element's font-size. One number for the whole page; follows the reader's setting (Chrome → Appearance → Font size).

em

on `font-size`: the parent's size. Elsewhere: this element's own.

the trap

a rule with `em` that matches nested elements multiplies itself. Cycle 3's scale is all `rem` for that reason.

Measured in cycle 3: `body` 17px, `h1` 33.12px.

UNITS

```
rem  x the ROOT font-size
em   x THIS element's font-size
%    of the containing block
ch   width of one "0"
ex   height of a lowercase x
vw   1% of viewport width
vh   1% of viewport height
vmin the smaller of vw and vh
px   one CSS pixel
fr   a share of free space (grid)
```

rem

type and spacing. Your default.

em

values that scale with this element's own text.

%

widths.

ch

measure, and nothing else.

px

borders, outlines, small radii.

vw / vh

viewport-sized things. Sparingly.

REFERENCE · NAMING THE DECISIONS

"THE CLIENT WANTS IT PURPLE"

demos/07-tokens/01-hardcoded – the same teal, six times

↶ reload

open ↷

Ruppín Coffee Club

Third floor, next to the printer that never works.

This week

Two coffees on the bar, both washed, both from producers we can actually name.

Ethiopia, Guji

Stone fruit, jasmine, and the faint taste of someone else paying.

Colombia, Huila

Caramel and red apple. The safe one, and there is no shame in that.

Joining

Turn up. Put your name on the rota if you want a key.

Run by second-years who should have been revising.

► **HANDS-ON** demos/07-tokens/01-hardcoded/ — Change it to purple live. Count the misses. Then: "and a bit more air between the sections."

THE PAYOFF, AND IT IS BIGGER THAN COLOUR

demos/07-tokens/03-themed — press the toggle, then read the stylesheet

↻ reload

open ↗

switch to dark

Ruppín Coffee Club

Third floor, next to the printer that never works.

This week

Two coffees on the bar, both washed, both from producers we can actually name.

Ethiopia, Guji

Stone fruit, jasmine, and the faint taste of someone else paying.

Colombia, Huila

Caramel and red apple. The safe one, and there is no shame in that.

Joining

Turn up. [Put your name on the rota](#) if you want a key.

Run by second-years who should have been revising.

► **HANDS-ON** demos/07-tokens/03-themed/ — Count the rules that changed below the token block. It is zero.

← SPINE · 23

YOU HAVE BEEN LOOKING AT ONE ALL MORNING

```
1 /* shared/tokens.css */  
2 --color-accent: hsl(234 89% 52%);  
3 --space-4: 1rem;  
4 --radius-xl: 28px;  
5 --font-display: 'Heebo', system-ui;  
6 --duration-normal: 220ms;
```

These slides, every demo you have seen today, and every Hebrew PDF in this course are drawn from one file with about forty tokens in it.

It is the same idea you are about to write, at the scale of a whole course. Open it — `shared/tokens.css` in the materials repo — and read the comments.

TOKENS: THE CONVENTION THIS COURSE USES

```
:root {  
  --color-bg: hsl(258 40% 99%);      /* the page ground          */  
  --color-surface: hsl(258 38% 96%); /* raised things          */  
  --color-border: hsl(258 20% 80%);  
  --color-text: hsl(258 30% 12%);  
  --color-text-muted: hsl(258 14% 36%);  
  --color-accent: hsl(258 72% 42%);  
  --color-accent-strong: hsl(258 76% 30%); /* the hover state    */  
  
  --space-1: 0.25rem; --space-4: 1rem;  
  --space-2: 0.5rem;  --space-5: 1.5rem;  
  --space-3: 0.75rem; --space-6: 2rem;  
  
  --radius-sm: 6px;    --radius-md: 12px;    --radius-full: 999px;  
  
  --font-sans: system-ui, sans-serif;  
  --font-size-base: 1.0625rem;  --font-size-lg: 1.33rem;  
  --font-size-sm: 0.85rem;      --font-size-xl: 1.66rem;  
  
  --duration: 150ms;  
}
```

DARK AND REDUCED MOTION: THE TWO QUERIES, COLOR-SCHEME, AND LIGHT-DARK()

```
/* 1. scrollbars and form controls follow suit */
:root { color-scheme: light dark; }

/* 2. your tokens change under the query */
@media (prefers-color-scheme: dark) {
  :root { --color-paper: hsl(170 20% 10%); }
}

/* 3. or both values in one call (Baseline 2024) */
--color-paper: light-dark(hsl(170 30% 98%),
                          hsl(170 20% 10%));

/* 4. motion: a token, not a blanket override */
@media (prefers-reduced-motion: reduce) {
  :root { --duration: 0s; }
}
```

Both queries read the operating system's settings; the browser reports them and you cannot set them from CSS. Test both in DevTools, Rendering, without touching the OS.

`light-dark()` needs `color-scheme` declared and works only inside colour values (Chrome 123, Firefox 120, Safari 17.5). The two-block form is what the assignment grades either way.

`prefers-contrast` and `forced-colors` are the next two to know. Not graded this year.

THE FIVE THAT WILL COST YOU TIME

WRONG

Specificity is a count of selectors.

`em` everywhere, including nested lists.

`width` is the width of the box.

`px` for type, so it never scales.

`!important` to win an argument.

RIGHT

Three numbers, compared column by column.

`rem` by default; `em` only when it should scale with this element.

Only with `box-sizing: border-box`. Otherwise padding and border are added on.

`rem`, so the reader's own setting is respected.

Lower the winner instead of raising the loser.

WHERE TO LOOK THINGS UP

MDN

`developer.mozilla.org` — the reference. Search `mdn css box-sizing` and take the first result.

CAN I USE

`caniuse.com` — is this safe to ship yet. Almost always yes, in 2027.

YOUR OWN DEVTOOLS

The fastest reference you own. Elements, then Computed, and every value the page actually resolved to — with the rule that set it.

Not Stack Overflow first. An answer from 2013 will tell you to use a float.